

SQL basics

From basic queries to window functions and joins.

Denis Boldin



What is SQL?

SQL is a language for working with data in relational databases. It lets you ask questions, filter, transform and combine data from different tables. You write a query, the database processes it and returns a result set.

Business question

Which artists are most popular?

SQL query

```
SELECT artist_name,  
COUNT(tracks) AS tracks  
FROM tracks  
GROUP BY artist_name  
ORDER BY COUNT(tracks) desc;
```

Result set

artist_name	tracks
Taylor Swift	120
Drake	95
The Weeknd	85

SQL dialects

SQL has a common core, but database systems add their own syntax, functions and extensions.

Database system	SQL dialect	What to know
PostgreSQL	PostgreSQL SQL	Reference dialect for this course
Greenplum	PostgreSQL dialect	PostgreSQL-based analytical DBMS
Microsoft SQL Server	T-SQL	Microsoft SQL extensions
MySQL	MySQL SQL	MySQL-specific syntax and functions
Oracle Database	Oracle SQL / PL/SQL	Oracle SQL + procedural PL/SQL

What we'll cover today

From basic queries to window functions and joins. With real examples.

1

Working with data

Dataset
Data types
Operators and functions

2

Filtering and sorting

WHERE
ORDER BY
LIMIT

3

Aggregates and groups

Aggregate functions
GROUP BY

4

Window functions

Running total
Ranking
Moving average

5

Subqueries and CTEs

Subqueries
CTEs
Real examples

6

Joins, unions and other stuff

JOINS
UNION
Transactions

7

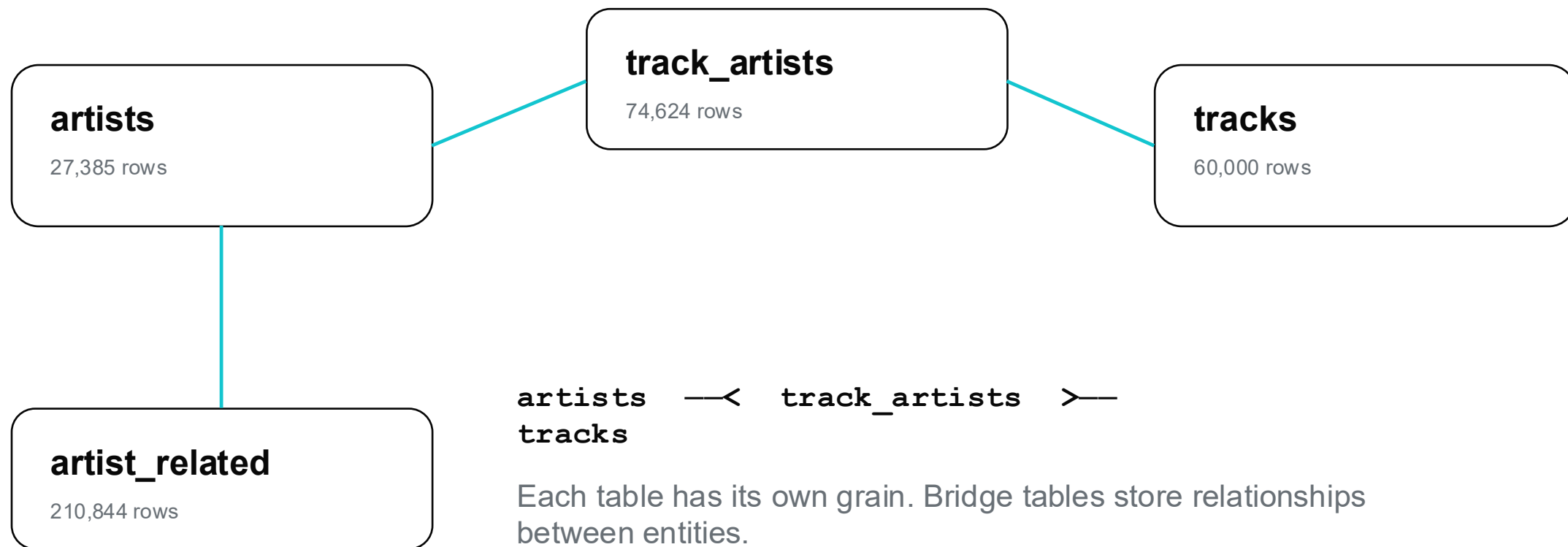
Wrap-up and Q&A

Key takeaways
Practice ideas
Your questions



The database we'll use

A Spotify catalogue sample, 1922–2021. We'll work with four related tables.





Data types affect query results

NUMERIC

INTEGER · BIGINT · REAL

TEXT

VARCHAR · TEXT

TIME

DATE · TIMESTAMP ·
INTERVAL

FLAG

BOOLEAN

```
duration_ms / 60000      -- 4
duration_ms / 60000.0    -- 4.175...
explicit::int            -- true → 1
```

Cast explicitly when the result depends on the data type.

```
CAST(duration_ms) as int
CAST(duration_ms) as varchar(100)

duration_ms :: int
```



Data types

Types matter because they:

- **ensure data integrity** — a numeric column will not accept string values;
- **optimize storage** — an integer takes less space than the same value as text;
- **optimize performance** — comparison and sorting are faster on numeric types than on character ones;
- **simplify working with data** — the schema becomes readable and self-documenting.

Types differ between DBMSs, so always check the documentation of the database you are working with.





Operators and functions

A **function** is a subroutine that takes arguments, performs an operation and returns a result. Functions can be nested — the result of one becomes the argument of the next.

Numeric operators and functions:

Operation	Operator	Examples
Addition	+	$3 + 2 \rightarrow 5$ · $3 + 2.0 \rightarrow 5.0$
Subtraction	-	$3 - 2 \rightarrow 1$
Multiplication	*	$3 * 2 \rightarrow 6$
Division	/	$3 / 2 \rightarrow 1$ · $3 / 2.0 \rightarrow 1.50$
Remainder	%	$3 \% 2 \rightarrow 1$
Exponentiation	^	$3 ^ 2 \rightarrow 9$
Absolute value	@	@ -3 → 3



Operators and functions

Purpose	Function	Examples
Remainder of division	<code>mod(numerator, denominator)</code>	<code>mod(5, 2) → 1</code>
Rounding	<code>round(number)</code>	<code>round(1.54) → 2</code>
Nearest larger integer	<code>ceil(number)</code>	<code>ceil(1.54) → 2</code>
Nearest smaller integer	<code>floor(number)</code>	<code>floor(1.54) → 1</code>
Exponentiation	<code>power(number, power)</code>	<code>power(3, 3) → 27</code>
Square root	<code>sqrt(number)</code>	<code>sqrt(1.5) → 1.22</code>



Operators and functions

If you use only integers, the result is an integer. If one argument is fractional, the result is fractional too.

That rule bites immediately on this data. `duration_ms / 60000` gives whole minutes and throws away the rest; `duration_ms / 60000.0` keeps the fraction:

```
SELECT name,  
       duration_ms,  
       duration_ms / 60000      AS minutes_int,  
       duration_ms / 60000.0    AS minutes_exact,  
       round(duration_ms / 60000.0, 2) AS minutes_rounded,  
       (duration_ms || 'milliseconds') :: interval AS interval,  
       to_char((duration_ms || 'milliseconds') :: interval, 'MI:SS') AS minutes_seconds_char  
FROM tracks  
where name = 'Everlong';
```

RESULT

name	duration_ms	minutes_int	minutes_exact	minutes_rounded	interval	minutes_seconds_char
Everlong	250547	4	4.175783333333333	4.18	00:04:10.547	04:10



Operators and functions

String functions:

Purpose	Function	Examples
Concatenation	<code>concat(a, b, c)</code>	<code>concat('Valya had ', 2, ' oranges') → 'Valya had 2 oranges'</code>
String length	<code>length(string)</code>	<code>length('123') → 3</code>
Trimming spaces	<code>trim(string)</code>	<code>trim(' text ') → 'text'</code>
Substring search	<code>position(substring in string)</code>	<code>position('4' in '123') → 0</code>
Upper / lower case	<code>upper(text), lower(text)</code>	<code>upper('text') → 'TEXT'</code>



Operators and functions

Date and time functions:

Purpose	Function	Examples
Current date and time	<code>now()</code>	<code>now()</code> → 2026-05-04 12:44:00
Addition	<code>date + number</code>	<code>date '2001-09-28' + 7</code> → <code>date '2001-10-05'</code>
Subtraction	<code>date - number</code>	<code>date '2001-09-28' - 7</code> → <code>date '2001-09-21'</code>
Difference	<code>date - date</code>	<code>date '2001-10-01' - date '2001-09-28'</code> → 3
Part of a date	<code>extract(part from date)</code>	<code>extract(year from date '2001-10-01')</code> → 2001



Operators and functions

`extract` is the workhorse of this lecture — it is how we get a year or a decade out of `release_date`:

```
SELECT name,  
       release_date,  
       EXTRACT(YEAR FROM release_date)::int           AS year,  
       (EXTRACT(YEAR FROM release_date)::int / 10) * 10 AS decade  
FROM tracks  
ORDER BY release_date  
LIMIT 5;
```

RESULT

name	release_date	year	decade
Maldita sea la primera vez	1900-01-01	1900	1900
Tu Verras Montmartre	1922-01-01	1922	1920
Lady of the Evening	1922-01-01	1922	1920
Nuits De Chine	1922-01-01	1922	1920
Marta	1922-01-01	1922	1920



Operators and functions

Boolean and comparison operators:

Purpose	Operator	Examples
Logical NOT / AND / OR	NOT, AND, OR	true AND false → false
Comparison	=, >, >=, <, <=, <>	7 > 5 → true
Range	BETWEEN	5 BETWEEN 3 AND 7 → true

Operator precedence: comparison operators are evaluated first, then NOT, then AND, and OR last.



The CASE operator

Bucketing tracks by how popular they are:

```
SELECT name,  
       popularity,  
       CASE WHEN popularity >= 80 THEN 'hit'  
            WHEN popularity >= 50 THEN 'known'  
            WHEN popularity >= 20 THEN 'deep cut'  
            ELSE 'obscure' END AS tier,  
       explicit,  
       explicit::int AS explicit_int  
FROM tracks  
ORDER BY popularity DESC, name  
LIMIT 6;
```

RESULT

name	popularity	tier	explicit	explicit_int
Peaches (feat. Daniel Caesar & Giveon)	100	hit	true	1
Blinking Lights	96	hit	false	0
WITHOUT YOU	94	hit	true	1
LA NOCHE DE ANOCHE	93	hit	false	0
DÁKITI	92	hit	true	1
BICHOTA	91	hit	true	1

Note `explicit::int` — a boolean cast to an integer gives 1 or 0, which is handy for summing later.



Filtering and sorting

- Filtering (WHERE)
- Filtering text
- Filtering by a computed field
- Sorting (ORDER BY)
- Limiting the result set (LIMIT, OFFSET)
- Missing data (NULL)
- Check yourself



Filtering (WHERE)

Several conditions are combined with **AND** and **OR**; **AND** binds tighter than **OR**.

```
SELECT name, popularity, release_date, explicit
FROM tracks
WHERE explicit
      AND popularity > 75
LIMIT 5;
```

RESULT

name	popularity	release_date	explicit
Creep	83	1993-02-22	true
Still D.R.E.	81	1999-11-16	true
All The Things She Said	76	2002-01-01	true
SexyBack (feat. Timbaland)	81	2006-09-12	true
Flashing Lights	78	2007-09-11	true

FILTERING DATA

Combine several filters in one **WHERE** clause

WHERE keeps only the rows that match your condition.

SQL

```
1 SELECT name, popularity, release_date
2 FROM tracks
3 WHERE popularity >= 80
4     AND explicit = false
5     AND release_date >= DATE '2010-01-01'
6     AND release_date < DATE '2020-01-01'
7 ORDER BY popularity DESC;
```

track_name	popularity	release_date
Blinding Lights	90	2019-11-29
Starboy	90	2016-09-21
Shape of You	90	2017-01-06
Believer	90	2017-02-01
HUMBLE.	90	2017-03-30



Text filters, NULLs, sorting and limits

Text = · LIKE · ILIKE

Missing IS NULL · IS NOT NULL · COALESCE

Sort ORDER BY ... ASC / DESC

Slice LIMIT · OFFSET

```
SELECT name, popularity
FROM tracks
WHERE name ILIKE '%love%'
ORDER BY popularity DESC NULLS LAST
LIMIT 5;
```

NULL ≠ 0 ≠ empty string

NULL means “unknown / absent”. Comparisons with = do not work the way you might expect.



Filtering text

ILIKE — the case-insensitive version (PostgreSQL). Track titles are typed by humans, so case is never reliable:

```
SELECT name, popularity
FROM tracks
WHERE name ILIKE '%love%'
ORDER BY popularity DESC
LIMIT 5;
```

RESULT

name	popularity
What You Know Bout Love	91
i love you	83
Another Love	83
Love The Way You Lie	83
Lasting Lover	83



Filtering text

IN — matches one of a list of values:

```
SELECT name, key, mode, time_signature
FROM tracks
WHERE time_signature IN (3, 5, 7)
AND popularity > 70
LIMIT 5;
```

RESULT

name	key	mode	time_signature
What A Wonderful World	5	1	3
Annie's Song	2	1	3
Somebody To Love - Remastered 2011	8	1	3
Crazy	6	0	3
Ironic - 2015 Remaster	11	1	5



1.1 FILTERING AND SORTING

Filtering text

Text functions in the filter — `LOWER()`, `UPPER()`, `TRIM()`, `SUBSTRING()`:

```
SELECT name
FROM tracks
WHERE LOWER(name) = 'yesterday'
LIMIT 3;
```

RESULT

name
Yesterday
Yesterday
Yesterday



Filtering text

💡 Wrapping a column in a function stops PostgreSQL using an ordinary index on it. `LOWER(name) = 'yesterday'` cannot use an index on `name`. That does not matter on 60,000 rows; it matters a great deal on 60,000,000. Lecture 3 returns to this.

Conditions combine freely:

```
SELECT name, popularity, tempo
FROM tracks
WHERE (name ILIKE '%love%' OR name ILIKE '%heart%')
AND popularity > 60
AND tempo BETWEEN 100 AND 130
LIMIT 5;
```

RESULT

name	popularity	tempo
Dedicated To The One I Love	65	103.264
Satellite of Love	61	121.861
Can't Get Enough Of Your Love, Babe	69	111.833
This Will Be (An Everlasting Love)	74	126.739
Say You Love Me	64	128.068



Sorting (ORDER BY)

The sort field can be given by name, by alias, or by its ordinal number among the output fields (starting from 1). Numbers sort numerically, character types lexicographically.

Unlike `WHERE`, `ORDER BY` can see a `SELECT` alias, because sorting happens after the columns are computed (see the order of execution):

```
SELECT name,  
       round(duration_ms / 60000.0, 2) AS minutes  
FROM tracks  
WHERE popularity > 70  
ORDER BY minutes DESC  
LIMIT 5;
```

RESULT

name	minutes
Poesia Acústica 10: Recomeçar	11.38
Poesia Acústica #6: Era uma Vez	9.63
Mirrors	8.07
Comfortably Numb	6.37
Sympathy For The Devil - 50th Anniversary Edition	6.30



Limiting the result set (LIMIT, OFFSET)

LIMIT restricts the number of rows returned; combined with **OFFSET** it gives pagination.

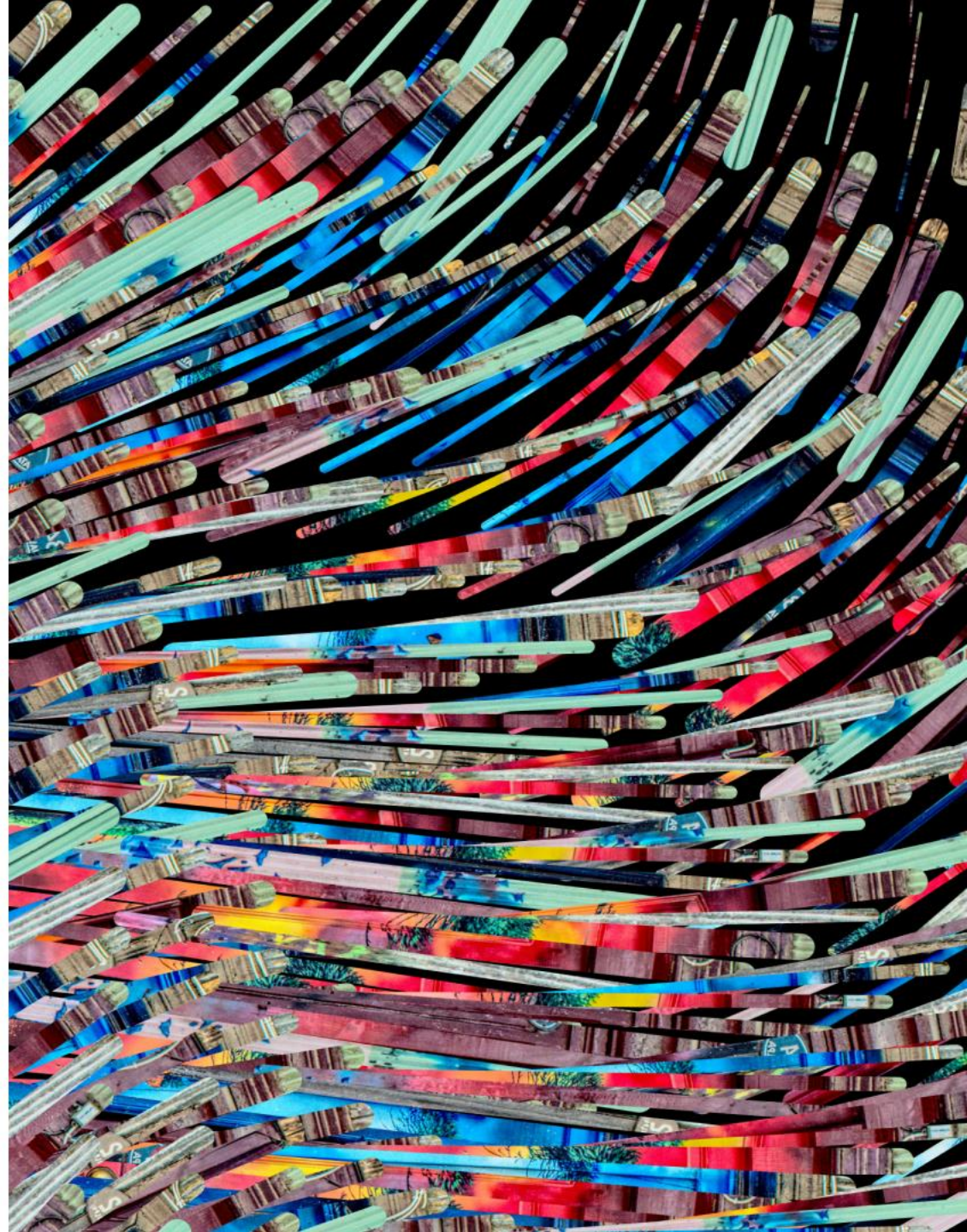
```
SELECT *  
  FROM table  
  WHERE <condition>  
ORDER BY <field_1>, <field_2>  
  LIMIT n1                -- return n1 rows  
  OFFSET n2              -- skip the first n2 rows
```



Aggregate functions

Aggregate functions perform a calculation over a set of values and return a single value — the aggregate. They are used to analyse and summarise data: the sum of today's purchases, the number of people in a department, the average length of a song.

- Specifics of COUNT
- Counting several things in one pass
- Grouping data (GROUP BY)
- Filtering by aggregates (HAVING)
- Check yourself





1.2 Aggregate functions

Purpose	Function	Example
Count	COUNT(<field>)	SELECT COUNT(*) FROM tracks
Sum	SUM(<field>)	SELECT SUM(duration_ms) FROM tracks
Average	AVG(<field>)	SELECT AVG(popularity) FROM tracks
Minimum	MIN(<field>)	SELECT MIN(release_date) FROM tracks
Maximum	MAX(<field>)	SELECT MAX(tempo) FROM tracks



1.2 Aggregate functions

```
SELECT count(*)           AS tracks,  
       round(avg(popularity), 2) AS avg_popularity,  
       min(release_date)   AS earliest,  
       max(release_date)   AS latest,  
       round(sum(duration_ms) / 3600000.0) AS total_hours  
FROM tracks;
```

RESULT

tracks	avg_popularity	earliest	latest	total_hours
60000	27.60	1900-01-01	2021-04-16	3827



Specifics of COUNT

- `COUNT(*)` — the number of rows in the table (or in the filtered table, if `WHERE` is used);
- `COUNT(1)` — the same as `COUNT(*)`, but the DBMS does not read the values of every field, using a constant instead;
- `COUNT(<field>)` — the number of non-empty values in the field;
- `COUNT(DISTINCT <field>)` — the number of unique non-empty values in the field.

The five nameless tracks make the difference visible:

```
SELECT count(*)           AS count_star,  
       count(1)           AS count_1,  
       count(name)        AS count_name,  
       count(DISTINCT name) AS count_distinct_name,  
       count(DISTINCT time_signature) AS count_distinct_time_sig  
FROM tracks;
```

RESULT

count_star	count_1	count_name	count_distinct_name	count_distinct_time_sig
60000	60000	59995	56448	5



Specifics of COUNT

`count_star` and `count_name` differ by exactly the five `NULL` titles. `count_distinct_name` is far lower still — the catalogue is full of remasters, live versions and re-releases sharing a title.





Grouping data (GROUP BY)

The single most useful grouping on this data is by decade:

```
SELECT (EXTRACT(YEAR FROM release_date)::int / 10) * 10 AS decade,  
       count(*) AS tracks,  
       round(avg(popularity), 1) AS avg_popularity,  
       round(avg(duration_ms) / 60000.0, 2) AS avg_minutes,  
       round(avg(energy)::numeric, 3) AS avg_energy  
FROM tracks  
GROUP BY decade  
ORDER BY decade;
```



Grouping data (GROUP BY)

RESULT

decade	tracks	avg_popularity	avg_minutes	avg_energy
1900	1	19.0	3.90	0.791
1920	778	1.1	2.99	0.285
1930	1333	2.0	3.54	0.308
1940	1845	1.8	3.65	0.271
1950	3617	8.3	3.68	0.296
1960	4834	18.2	3.48	0.407
1970	6325	24.2	3.91	0.503
1980	8419	25.9	3.81	0.549
1990	11135	29.3	4.01	0.571
2000	8881	36.6	4.05	0.648
2010	10764	39.3	3.86	0.662
2020	2068	41.1	3.33	0.634



Filtering by aggregates (HAVING)

```
SELECT a.name,  
       count(*) AS tracks  
FROM artists AS a  
JOIN track_artists AS ta ON ta.artist_id = a.id  
GROUP BY a.name  
HAVING count(*) > 150  
ORDER BY tracks DESC;
```

RESULT

name	tracks
Die drei ???	392
Lata Mangeskar	282
Francisco Canaro	210
TKKG Retro-Archiv	210
Johann Sebastian Bach	195
Wolfgang Amadeus Mozart	188
Bibi Blocksberg	163
Benjamin Blümchen	163

10-minute break

We'll continue with
window functions.





Window functions

An **analytic (window) function** performs calculations over a set of records forming a window (partition) and returns a single value per row.

- Syntax
- The base query for the examples
- The three groups of analytic functions
- Aggregate window functions
- Offset functions
- Ranking functions
- Frames
- WINDOW — a named window
- Check yourself

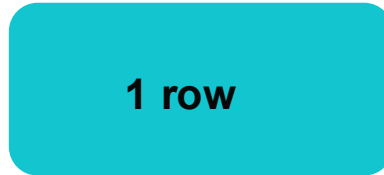


GROUP BY vs window functions

GROUP BY



4 rows



WINDOW





Syntax

```
<function>(<field>) OVER (PARTITION BY <partition> ORDER BY <sorting> <frame>)
```

- **function** — the window function over the chosen field;
- **partition** — the field or set of fields defining the group;
- **sorting** — the field or fields by which rows are ordered inside the partition;
- **frame** — the set of rows inside the window we operate on (optional).

First the data is split into partitions, then sorted, and only then is the function applied to each record inside the partition.



The three groups of analytic functions

AGGREGATION	OFFSET	RANKING
AVG()	FIRST_VALUE()	ROW_NUMBER()
COUNT()	LAST_VALUE()	RANK()
MAX()	LAG()	DENSE_RANK()
MIN()	LEAD()	NTILE()
SUM()	NTH_VALUE()	CUME_DIST()
		PERCENT_RANK()



Aggregate window functions

AVG, COUNT, MAX, MIN, SUM — cumulative values inside a window:

```
WITH artist_decade AS (  
    SELECT a.name AS artist,  
           (EXTRACT(YEAR FROM t.release_date)::int / 10) * 10 AS decade,  
           count(*) AS track_cnt  
    FROM artists AS a  
    JOIN track_artists AS ta ON ta.artist_id = a.id  
    JOIN tracks AS t ON t.id = ta.track_id  
    WHERE a.name IN ('David Bowie', 'Bob Dylan')  
    GROUP BY 1, 2  
)  
SELECT artist,  
       decade,  
       track_cnt,  
       SUM(track_cnt) OVER w ::int           AS running_sum,  
       round(AVG(track_cnt) OVER w, 2)       AS running_avg,  
       MIN(track_cnt) OVER w ::int           AS running_min,  
       MAX(track_cnt) OVER w ::int           AS running_max  
FROM artist_decade  
WINDOW w AS (PARTITION BY artist ORDER BY decade)  
ORDER BY artist, decade;
```



Aggregate window functions

RESULT

artist	decade	track_cnt	running_sum	running_avg	running_min	running_max
Bob Dylan	1960	22	22	22.00	22	22
Bob Dylan	1970	16	38	19.00	16	22
Bob Dylan	1980	4	42	14.00	4	22
David Bowie	1960	3	3	3.00	3	3
David Bowie	1970	14	17	8.50	3	14
David Bowie	1980	6	23	7.67	3	14
David Bowie	1990	1	24	6.00	1	14
David Bowie	2010	8	32	6.40	1	14



Offset functions

Offset functions reach into neighbouring rows:

- **FIRST_VALUE** — the first row in the frame
- **LAST_VALUE** — the last row in the frame
- **LAG** — the value in the previous row of the partition
- **LEAD** — the value in the next row of the partition
- **NTH_VALUE** — the value in row number N (the second argument)

```
WITH artist_decade AS (  
    SELECT a.name AS artist,  
           (EXTRACT(YEAR FROM t.release_date)::int / 10) * 10 AS decade,  
           count(*) AS track_cnt  
    FROM artists AS a  
    JOIN track_artists AS ta ON ta.artist_id = a.id  
    JOIN tracks AS t ON t.id = ta.track_id  
    WHERE a.name IN ('David Bowie', 'Bob Dylan')  
    GROUP BY 1, 2  
)  
SELECT artist,  
       decade,  
       track_cnt,  
       FIRST_VALUE(track_cnt) OVER w AS first_decade,  
       LAG(track_cnt) OVER w AS prev_decade,  
       LEAD(track_cnt) OVER w AS next_decade,  
       track_cnt - LAG(track_cnt) OVER w AS change,  
       NTH_VALUE(track_cnt, 2) OVER w AS second_decade  
FROM artist_decade  
WINDOW w AS (PARTITION BY artist ORDER BY decade)  
ORDER BY artist, decade;
```



Offset functions

RESULT

artist	decade	track_cnt	first_decade	prev_decade	next_decade	change	second_decade
Bob Dylan	1960	22	22	NULL	16	NULL	NULL
Bob Dylan	1970	16	22	22	4	-6	16
Bob Dylan	1980	4	22	16	NULL	-12	16
David Bowie	1960	3	3	NULL	14	NULL	NULL
David Bowie	1970	14	3	3	6	11	14
David Bowie	1980	6	3	14	1	-8	14
David Bowie	1990	1	3	6	8	-5	14
David Bowie	2010	8	3	1	NULL	7	14

LAG is how you compute any change-over-time: the value minus the previous value. Note the **NULL** in the first row of each partition — there is no previous row, so the difference is unknown, not zero.



Ranking functions

- **ROW_NUMBER** — the row number inside the partition
- **RANK** — the rank within the partition, skipping the next rank when values repeat
- **DENSE_RANK** — the rank without gaps when values repeat
- **NTILE** — splits the window into the given number of groups and returns the group number
- **CUME_DIST** — the cumulative distribution, RANK/COUNT
- **PERCENT_RANK** — the relative rank, $(RANK-1)/(COUNT-1)$

Ties are what separate these functions, and popularity scores are full of ties:

```
WITH bowie AS (  
    SELECT t.name, t.popularity  
    FROM tracks AS t  
    JOIN track_artists AS ta ON ta.track_id = t.id  
    JOIN artists AS a ON a.id = ta.artist_id  
    WHERE a.name = 'David Bowie'  
    ORDER BY t.popularity DESC, t.name  
    LIMIT 8  
)  
SELECT name,  
    popularity,  
    ROW_NUMBER() OVER w AS rn,  
    RANK() OVER w AS rank,  
    DENSE_RANK() OVER w AS dense_rank,  
    NTILE(2) OVER w AS ntile_2,  
    round((CUME_DIST() OVER w)::numeric, 2) AS cume_dist,  
    round((PERCENT_RANK() OVER w)::numeric, 2) AS percent_rank  
FROM bowie  
WINDOW w AS (ORDER BY popularity DESC);
```



Ranking functions

RESULT

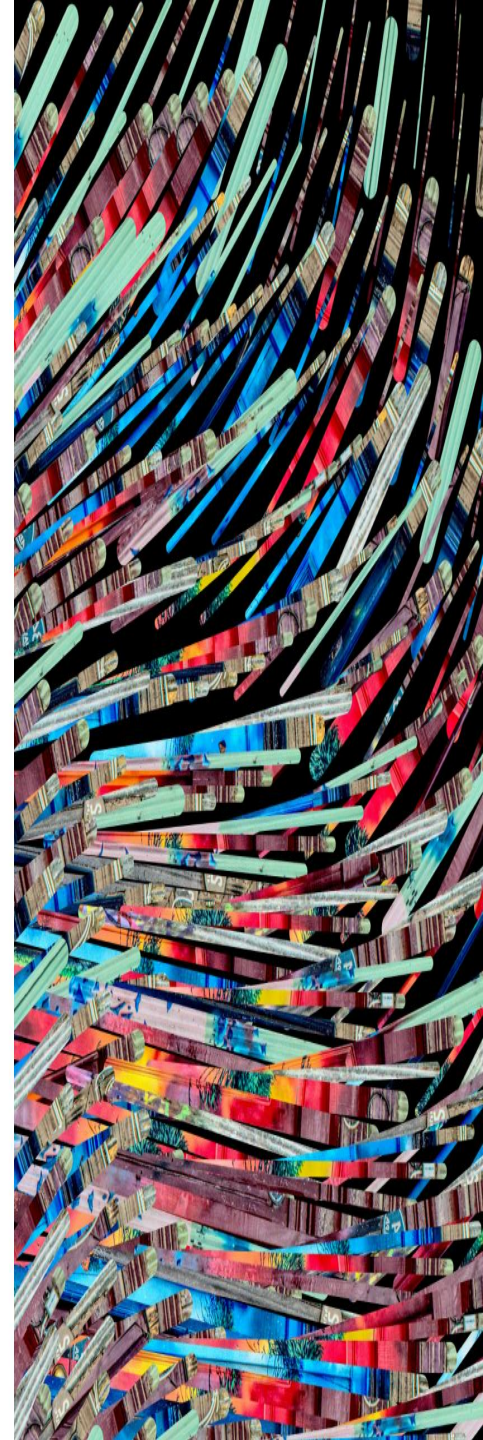
name	popularity	rn	rank	dense_rank	ntile_2	cume_dist	percent_rank
Moonage Daydream - 2012 Remaster	69	1	1	1	1	0.13	0.00
Sound and Vision - 2017 Remaster	63	2	2	2	1	0.25	0.14
Under Pressure	59	3	3	3	1	0.38	0.29
As The World Falls Down	55	4	4	4	1	0.50	0.43
Tonight (With David Bowie) - Live	48	5	5	5	2	0.63	0.57
Cracked Actor - 2013 Remaster	43	6	6	6	2	0.88	0.71
Word on a Wing - 2016 Remaster	43	7	6	6	2	0.88	0.71
Beauty and the Beast - 2017 Remaster	42	8	8	7	2	1.00	1.00



1.4

Subqueries and CTEs

Use them when a query is easier to solve in steps.





Subqueries: use one query inside another

Find tracks more popular than the catalogue average.

```
SELECT name, popularity
FROM tracks
WHERE popularity > (
    SELECT avg(popularity)
    FROM tracks
)
ORDER BY popularity DESC;
```

1. inner query

average popularity



2. outer query

compare every
track

Correlated subqueries run with values from the outer row; scalar subqueries return one value.

CTEs: split a query into readable steps

CTEs (Common Table Expressions) let you break a query into smaller parts, making it easier to read and maintain.

SQL

```
WITH top_artists AS (  
  SELECT artist_name, AVG(popularity) AS avg_pop  
  FROM tracks  
  GROUP BY artist_name  
  HAVING AVG(popularity) > 70  
)  
SELECT *  
FROM top_artists  
ORDER BY avg_pop DESC;
```





Common Table Expression (CTE)

Advantages:

1. **Improved readability** — a complex query is split into understandable parts.
2. **Reuse** — the CTE's result can be referenced several times without repeating the text of the subquery.

There can be several CTEs in one query: **WITH** is written once and the CTEs are listed separated by commas. A later CTE may reference an earlier one:

```
WITH artist_tracks AS (  
    SELECT a.id, a.name, t.popularity  
    FROM artists AS a  
    JOIN track_artists AS ta ON ta.artist_id = a.id  
    JOIN tracks AS t        ON t.id = ta.track_id  
)  
artist_stats AS (  
    SELECT id,  
           name,  
           count(*)           AS track_cnt,  
           round(avg(popularity), 1) AS avg_popularity  
    FROM artist_tracks  
    GROUP BY id, name  
)  
SELECT name, track_cnt, avg_popularity  
FROM artist_stats  
WHERE track_cnt >= 30  
ORDER BY avg_popularity DESC  
LIMIT 10;
```



Joins and unions

- DISTINCT
- UNION and UNION ALL
- JOIN
- Joins on the real data
- Special types of JOIN
- Venn diagrams
- Joining and aggregating together
- Check yourself



DISTINCT

DISTINCT removes duplicate rows from the query result:

```
SELECT DISTINCT column1 FROM table_name;  
SELECT DISTINCT column1, column2 FROM table_name;    -- unique combinations
```

DISTINCT is written once, right after **SELECT**. The DBMS has to remember every unique value and compare it with the following ones, which can slow the query down.



UNION and UNION ALL

UNION and **UNION ALL** combine the results of two or more queries **vertically**:

- **UNION** combines the results, excluding duplicate rows;
- **UNION ALL** does the same but keeps duplicates.



JOINS

Combining tables with JOIN

JOIN lets you combine related data from multiple tables using a matching key.

tracks

track_id

track_name

artist_id

album_id

popularity

JOIN

ON tracks.artist_id
= artists.artist_id

artists

artist_id

artist_name

followers

genres





JOIN

JOIN enriches data from one table with data from another — a **horizontal** combination. Every row in one table is matched against the second by some condition, and matching rows are combined.

```
SELECT a.column1, b.column2
FROM table1 a
JOIN table2 b ON a.id = b.table2_id;
```

Tables can be given aliases so that you do not have to write their names out in full; **AS** may be omitted here.



JOIN

INNER JOIN returns the rows that have a match in both tables:

```
WITH releases(release_id, title, label_id) AS (VALUES
    (1, 'Space Oddity', 1), (2, 'Heroes', 2), (3, 'Blackstar', 9),
    (4, 'Blonde on Blonde', 3), (5, 'Bedroom Demo', NULL), (6, 'Lodger', 2)
),
labels(label_id, label_name) AS (VALUES
    (1, 'Philips'), (2, 'RCA'), (3, 'Columbia'), (4, 'Sub Pop')
)
SELECT r.release_id, r.title, l.label_name
FROM releases AS r
INNER JOIN labels AS l ON r.label_id = l.label_id
ORDER BY r.release_id;
```

RESULT

release_id	title	label_name
1	Space Oddity	Philips
2	Heroes	RCA
4	Blonde on Blonde	Columbia
6	Lodger	RCA



1.5 JOINS AND UNIONS

JOIN

LEFT JOIN returns all rows from the left table and the matching rows from the right one; where there is no match, **NULL**:

```
WITH releases(release_id, title, label_id) AS (VALUES
    (1, 'Space Oddity', 1), (2, 'Heroes', 2), (3, 'Blackstar', 9),
    (4, 'Blonde on Blonde', 3), (5, 'Bedroom Demo', NULL), (6, 'Lodger', 2)
),
labels(label_id, label_name) AS (VALUES
    (1, 'Philips'), (2, 'RCA'), (3, 'Columbia'), (4, 'Sub Pop')
)
SELECT r.release_id, r.title, l.label_name
FROM releases AS r
LEFT JOIN labels AS l ON r.label_id = l.label_id
ORDER BY r.release_id;
```

RESULT

release_id	title	label_name
1	Space Oddity	Philips
2	Heroes	RCA
3	Blackstar	NULL
4	Blonde on Blonde	Columbia
5	Bedroom Demo	NULL
6	Lodger	RCA



JOIN

RIGHT JOIN is the mirror image — all rows from the right table plus matches from the left. It shows which labels exist even if they have no releases:

```
WITH releases(release_id, title, label_id) AS (VALUES
    (1, 'Space Oddity', 1), (2, 'Heroes', 2), (3, 'Blackstar', 9),
    (4, 'Blonde on Blonde', 3), (5, 'Bedroom Demo', NULL), (6, 'Lodger', 2)
),
labels(label_id, label_name) AS (VALUES
    (1, 'Philips'), (2, 'RCA'), (3, 'Columbia'), (4, 'Sub Pop')
)
SELECT r.release_id, r.title, l.label_name
FROM releases AS r
RIGHT JOIN labels AS l ON r.label_id = l.label_id
ORDER BY l.label_name;
```

RESULT

release_id	title	label_name
4	Blonde on Blonde	Columbia
1	Space Oddity	Philips
2	Heroes	RCA
6	Lodger	RCA
NULL	NULL	Sub Pop



1.5 JOINS AND UNIONS

JOIN

FULL JOIN returns all rows from both tables, filling the missing values with **NULL** — every release *and* every label, matched where possible:

```
WITH releases(release_id, title, label_id) AS (VALUES
    (1, 'Space Oddity', 1), (2, 'Heroes', 2), (3, 'Blackstar', 9),
    (4, 'Blonde on Blonde', 3), (5, 'Bedroom Demo', NULL), (6, 'Lodger', 2)
),
labels(label_id, label_name) AS (VALUES
    (1, 'Philips'), (2, 'RCA'), (3, 'Columbia'), (4, 'Sub Pop')
)
SELECT r.release_id, r.title, l.label_name
FROM releases AS r
FULL JOIN labels AS l ON r.label_id = l.label_id
ORDER BY r.release_id NULLS LAST;
```

RESULT

release_id	title	label_name
1	Space Oddity	Philips
2	Heroes	RCA
3	Blackstar	NULL
4	Blonde on Blonde	Columbia
5	Bedroom Demo	NULL
6	Lodger	RCA
NULL	NULL	Sub Pop



Special types of JOIN

CROSS JOIN — a join without a condition, producing the Cartesian product of two tables. Useful when every row of one table must be matched with every row of another, but dangerous: the row count grows multiplicatively.

```
WITH top3 AS (  
    SELECT name FROM tracks ORDER BY popularity DESC, name LIMIT 3  
)  
SELECT a.name AS track_1, b.name AS track_2  
    FROM top3 AS a  
    CROSS JOIN top3 AS b  
    WHERE a.name <> b.name  
    ORDER BY 1, 2;
```

RESULT

track_1	track_2
Blinding Lights	Peaches (feat. Daniel Caesar & Giveon)
Blinding Lights	WITHOUT YOU
Peaches (feat. Daniel Caesar & Giveon)	Blinding Lights
Peaches (feat. Daniel Caesar & Giveon)	WITHOUT YOU
WITHOUT YOU	Blinding Lights
WITHOUT YOU	Peaches (feat. Daniel Caesar & Giveon)



Special types of JOIN

Three rows crossed with three rows give nine, less the three where a track meets itself. Do this to `tracks` unfiltered and you ask for 3.6 billion rows.

SELF JOIN — joining a table with itself. `artist_related` links artists to artists, so reading it means joining `artists` twice:

```
SELECT a.name AS artist,  
       r.name AS also_liked  
FROM artist_related AS ar  
JOIN artists AS a ON a.id = ar.artist_id  
JOIN artists AS r ON r.id = ar.related_artist_id  
WHERE a.name = 'David Bowie'  
ORDER BY ar.position  
LIMIT 8;
```

RESULT

artist	also_liked
David Bowie	Lou Reed
David Bowie	T. Rex
David Bowie	Roxy Music
David Bowie	Iggy Pop
David Bowie	Talking Heads
David Bowie	Brian Eno
David Bowie	Queen
David Bowie	The Velvet Underground



Special types of JOIN

Anti-join — not an official join type, but a common pattern: rows in one table with no match in another. Our sample has 1,099 tracks whose credited artist was missing from the source, and this is how you find them:

```
SELECT t.name, t.release_date, t.popularity
FROM tracks AS t
LEFT JOIN track_artists AS ta ON ta.track_id = t.id
WHERE ta.track_id IS NULL
ORDER BY t.popularity DESC
LIMIT 5;
```

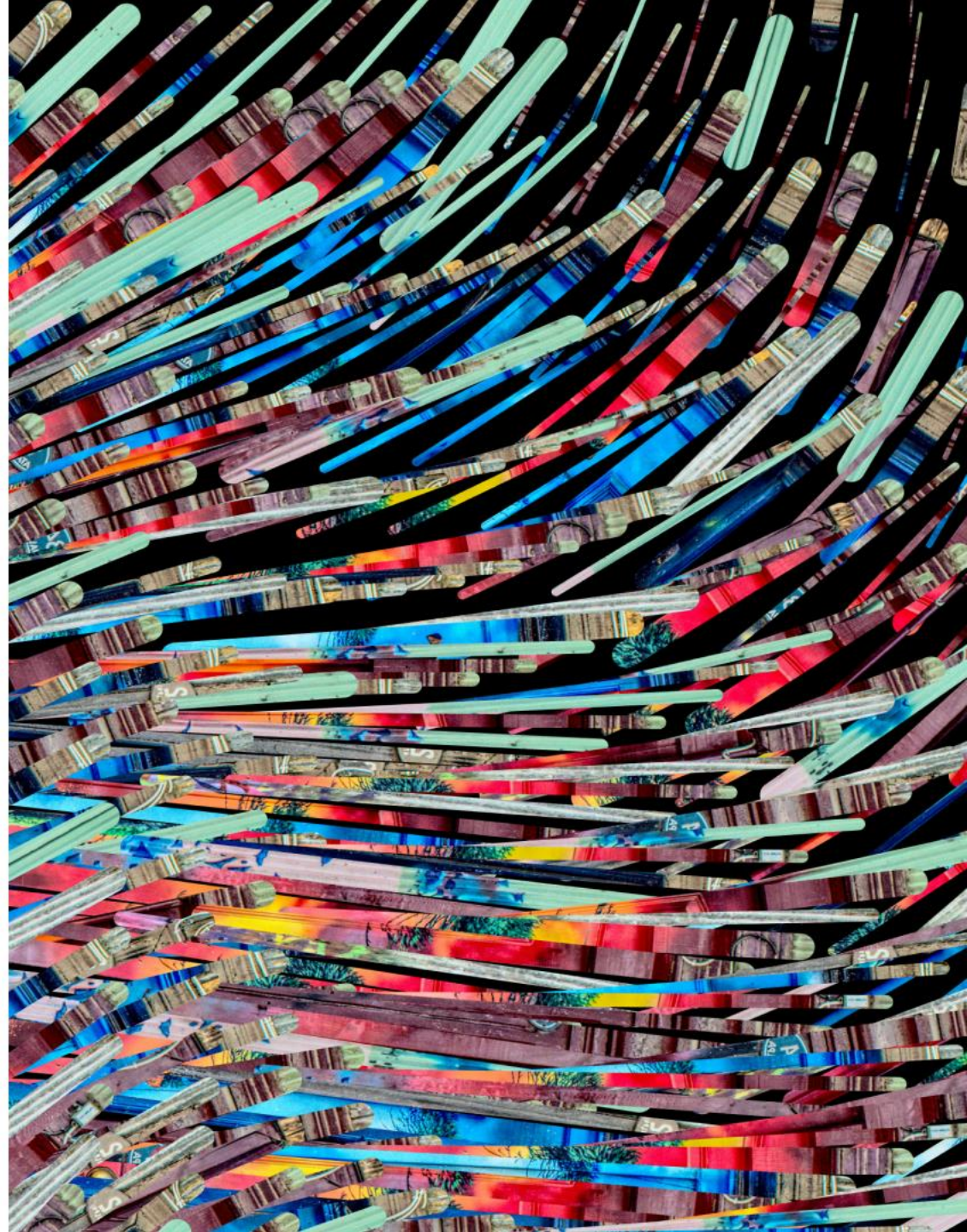
RESULT

name	release_date	popularity
Body Like A Back Road	2017-02-01	76
So Pretty	2021-01-20	73
Following The Sun	2020-10-02	71
Idk	2020-07-23	64
Big Fan Dulled	2017-05-07	64



ACID and the execution plan

- ACID
- Order of execution of clauses
- The execution plan





ACID

ACID is the set of four guarantees a transactional DBMS gives:

- **Atomicity** — a transaction is all-or-nothing. There is no state in which half of its statements have been applied.
- **Consistency** — a transaction moves the database from one valid state to another; all constraints (primary and foreign keys, `CHECK`, `NOT NULL`) hold before and after it. Our `track_artists.artist_id` has a foreign key to `artists`, so the database will simply refuse a credit for an artist that does not exist.
- **Isolation** — concurrent transactions do not see each other's intermediate results. How strictly is governed by the **isolation level**: `READ UNCOMMITTED`, `READ COMMITTED` (the PostgreSQL default), `REPEATABLE READ`, `SERIALIZABLE`. The stricter the level, the fewer the anomalies (dirty reads, non-repeatable reads, phantom reads) and the higher the cost in concurrency.
- **Durability** — once `COMMIT` returns, the data survives a crash or a power failure, because it has been written to the transaction log on disk.



Order of execution of clauses

We write an SQL query in one order and the DBMS executes it in another. The logical order of execution is:

1. **FROM** — the engine determines which tables the data will be taken from.
2. **ON** — the join conditions are processed; filtering happens before the join itself.
3. **JOIN** — the tables are joined on the specified conditions.
4. **WHERE** — the rows satisfying the query's conditions are filtered.
5. **GROUP BY** — the rows are grouped by the specified columns.
6. **HAVING** — the aggregated data is filtered; groups that fail the condition are cut off.
7. **SELECT** — the columns given in the query are computed and extracted.
8. **DISTINCT** — duplicates are removed from the result, if required.
9. **ORDER BY** — the data is sorted.
10. **LIMIT / OFFSET** — the rows for output are finally determined.



The execution plan

The logical order above says *what* the DBMS computes. **The execution plan** says *how* — which physical operations it will perform, in what order, and at what cost. The plan is produced by the optimizer, which analyses table statistics and chooses access methods and join algorithms.

`EXPLAIN` shows the plan without running the query:

EXPLAIN

```
SELECT a.name, count(*) AS tracks
  FROM artists AS a
 JOIN track_artists AS ta ON ta.artist_id = a.id
GROUP BY a.name
ORDER BY tracks DESC
LIMIT 10;
```

RESULT

QUERY PLAN

Limit (cost=4007.86..4007.88 rows=10 width=21)
-> Sort (cost=4007.86..4075.91 rows=27220 width=21)
Sort Key: (count(*)) DESC
-> HashAggregate (cost=3147.44..3419.64 rows=27220 width=21)
Group Key: a.name
-> Hash Join (cost=1112.16..2774.32 rows=74624 width=13)
Hash Cond: (ta.artist_id = a.id)
-> Seq Scan on track_artists ta (cost=0.00..1466.24 rows=74624 width=23)
-> Hash (cost=769.85..769.85 rows=27385 width=36)
-> Seq Scan on artists a (cost=0.00..769.85 rows=27385 width=36)

WRAP-UP

How to approach a SQL query

Before writing SQL, define what the result should represent.

1

Population

Which rows do I need?

WHERE

2

Grain

What does one row represent?

GROUP BY

3

Metric

What do I calculate?

AVG /
COUNT

4

Context

Do rows need comparison?

WINDOW

5

Structure

Would steps make it clearer?

CTE

6

Sanity

Could NULLs or duplicates break it?

CHECK

Good SQL starts with a clear definition of the rows, grain and metric you need.



Summary

Filtering and sorting. `WHERE` selects rows by a condition; `AND` and `OR` combine conditions, with `AND` binding tighter — set precedence explicitly with parentheses. Text is filtered with `=`, `LIKE`, `ILIKE`, `IN` and string functions. `ORDER BY` sorts (`ASC/DESC`), `LIMIT` and `OFFSET` paginate. `NULL` means the absence of data: compare it with `IS NULL` / `IS NOT NULL`, replace it with `COALESCE`, and place it explicitly in sorts with `NULLS FIRST` / `NULLS LAST`.

Aggregate functions. `SUM`, `MIN`, `MAX`, `AVG`, `COUNT` compute a single value over a set of rows. `COUNT(*)` counts rows, `COUNT(field)` counts non-`NULL` values. `GROUP BY` applies aggregates per group, `HAVING` filters the groups afterwards, and `FILTER (WHERE ...)` conditions a single aggregate. Filter non-aggregated data in `WHERE`, before grouping.

Window functions. `<function>(<field>) OVER (PARTITION BY ... ORDER BY ... <frame>)` computes over a window while returning every input row. Three groups: aggregate (`SUM`, `AVG`, `MIN`, `MAX`, `COUNT`), offset (`LAG`, `LEAD`, `FIRST_VALUE`, `LAST_VALUE`, `NTH_VALUE`) and ranking (`ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`, `CUME_DIST`, `PERCENT_RANK`). Frames (`ROWS`, `RANGE`) fix the exact set of rows; `WINDOW` names a window for reuse. To filter on a window function, compute it in a CTE and filter outside.



Summary

Subqueries and CTE. A subquery is a query inside another query — non-correlated, correlated or scalar — usable in `SELECT`, `FROM`, `JOIN`, `WHERE`, `HAVING` and `ORDER BY`, with `IN`, `EXISTS`, `ANY/SOME` and `ALL`. A CTE (`WITH`) lifts a subquery out of the main query, improving readability and letting one result be reused. `WITH RECURSIVE` walks graphs and hierarchies. A CTE lives only for the duration of the query; a temporary table lives until the end of the session.

Joins and unions. `DISTINCT` removes duplicate rows. `UNION/UNION ALL` combine results vertically (`UNION` deduplicates, `UNION ALL` is faster). `JOIN` combines them horizontally: `INNER`, `LEFT`, `RIGHT`, `FULL`, `CROSS`, plus the `SELF JOIN` and anti-join patterns. A many-to-many needs a bridge table — here, `track_artists`.

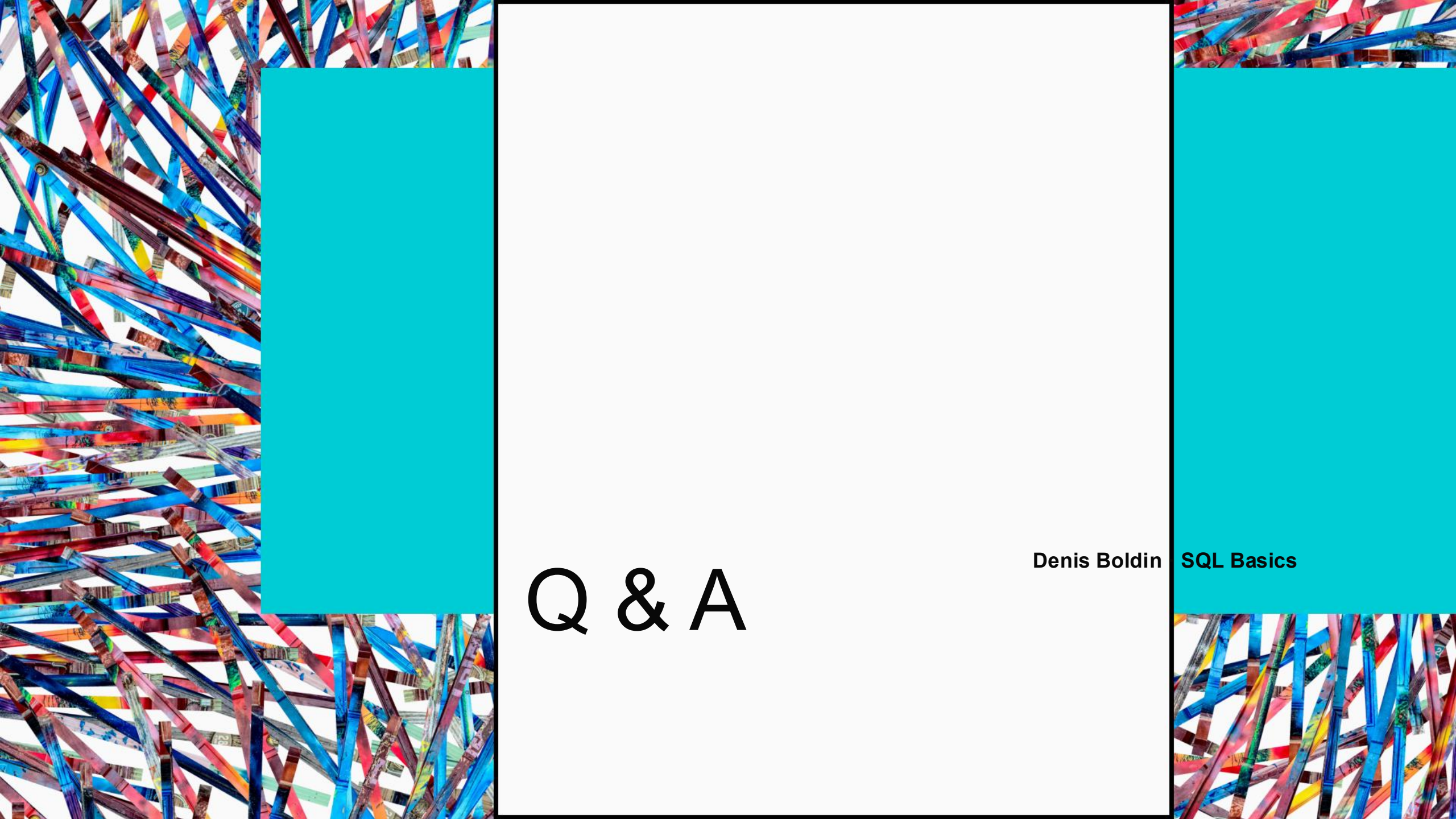


Summary

ACID and execution. Transactions guarantee Atomicity, Consistency, Isolation and Durability. Clauses execute in the order `FROM` → `ON` → `JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT/OFFSET`; `EXPLAIN` shows the physical plan the optimizer chose.

By the end of this lecture you can write an SQL query using the following clauses:

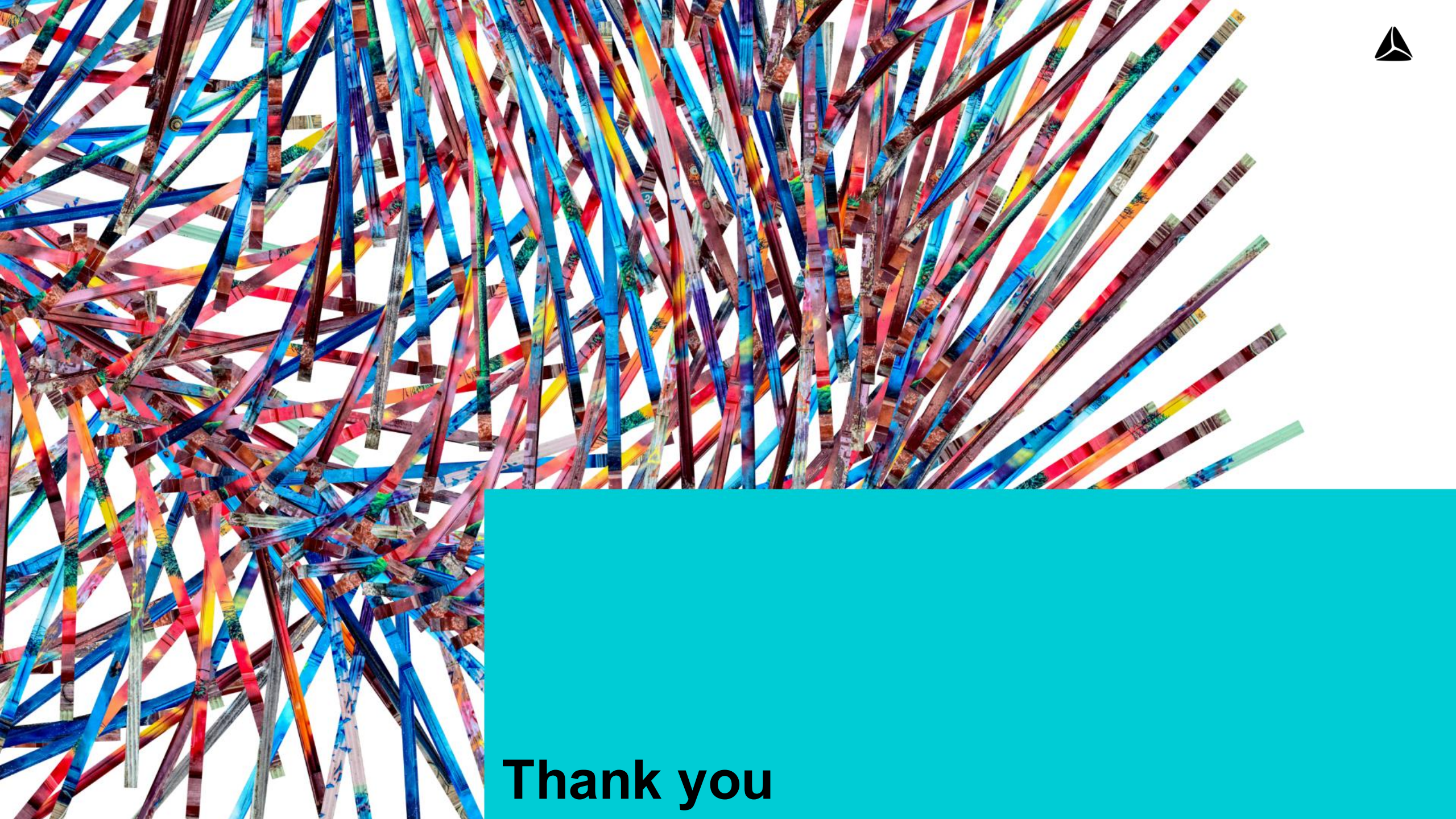
Clause	Description
<code>SELECT</code>	Queries data from one or several tables; selects specific columns and expressions
<code>DISTINCT</code>	Removes duplicate rows from the query result
<code>FROM</code>	Points to the table (or tables) the query addresses
<code>JOIN</code>	Joins rows from two or more tables on a related column: <code>INNER</code> , <code>LEFT</code> , <code>RIGHT</code> , <code>FULL</code> , <code>CROSS</code>
<code>WHERE</code>	Specifies the conditions by which rows are selected
<code>GROUP BY</code>	Groups rows with the same values in the specified columns for aggregation
<code>HAVING</code>	Filters the groups created by <code>GROUP BY</code>
<code>OVER / WINDOW</code>	Defines the window a window function is computed over
<code>WITH</code>	Declares a CTE — a named subquery used within the query; <code>WITH RECURSIVE</code> for graphs
<code>ORDER BY</code>	Orders the rows of the result (<code>ASC</code> / <code>DESC</code>)
<code>LIMIT</code> and <code>OFFSET</code>	<code>LIMIT</code> restricts the number of rows returned; <code>OFFSET</code> skips rows before returning the rest
<code>UNION</code> and <code>UNION ALL</code>	Combine the results of several <code>SELECT</code> queries; <code>UNION</code> removes duplicates



Q & A

Denis Boldin

SQL Basics



Thank you